

PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

**Enhancing Debugging and Maintainability of ASP.NET Core Backend APIs
submitted in partial fulfilment of the requirements for the award of degree of**

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Tanvi Gupta (2210990905)

Vanshita (2210992510)

Supervised By:

Dr Rajat Takkar

Supervisor

Chitkara University



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	
	List of Figures and Tables	
	Abstract	
1	Introduction	6
2	Methodology	7
3	Tools and Technologies	8
4	Implementation	9
5	Major Findings/Outcomes/Output/Results	10
6	Conclusion and Future Scope	11
	References	
	Appendices	

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “Enhancing Debugging and Maintainability of ASP.NET Core Backend APIs” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr Rajat Takkar. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

(Tanvi Gupta, Vanshita)

Date: 15-05-2026

(University Roll No: 2210990905, 2210992510)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

(Dr Rajat Takkar)

Supervisor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr Rajat Takkar for his valuable guidance, encouragement, and support throughout the completion of this CO-OP Project at Industry (Module-2). His constant supervision helped us understand the importance of building maintainable backend systems and applying systematic approaches to debugging and error handling.

We are also thankful to the Department of Computer Science and Engineering, Chitkara University, Punjab, for providing us with the opportunity to work on a practical project related to ASP.NET Core backend APIs. This project helped us strengthen our understanding of structured logging, centralized exception handling, API response consistency, and backend maintainability.

Lastly, we extend our gratitude to our mentors, faculty members, and peers for their support and feedback during the development and presentation of this project.

ABSTRACT

Backend APIs are responsible for handling the core operations of modern applications, including data processing, business logic, database communication, and service responses. As backend systems grow in size and complexity, debugging becomes more difficult when logs are unstructured and exception handling is scattered across multiple parts of the application. This often results in increased debugging time, repeated try-catch blocks, inconsistent API responses, and reduced maintainability.

This project, titled "Enhancing Debugging and Maintainability of ASP.NET Core Backend APIs", focuses on improving backend API reliability by implementing structured logging and centralized exception handling. Structured logging stores logs in a meaningful key-value format, making it easier to trace errors and understand request flow. Centralized exception handling uses middleware to manage errors globally, reducing code duplication and ensuring consistent responses.

The project compares a traditional backend system with a proposed improved system. Testing was performed using invalid inputs, database failure scenarios, and API exceptions. The proposed approach showed improved traceability, cleaner code, better debugging efficiency, and enhanced maintainability. The work concludes that structured logging and centralized exception handling are effective techniques for building scalable and maintainable ASP.NET Core backend APIs.

1. Introduction

1.1 Overview

Backend APIs play a major role in modern software systems. They handle the communication between the client-side application, the database, and other services. Every request made by a user is processed by backend APIs, which perform validations, execute business logic, interact with databases, and return appropriate responses.

As applications become larger, debugging backend APIs becomes more challenging. Developers often need to identify the exact cause of failures from logs, exception messages, request details, and database responses. If logging is not structured properly, it becomes difficult to trace the issue. Similarly, if exception handling is written separately in multiple controllers or services, the code becomes repetitive and harder to maintain.

1.2 Problem Statement

In traditional backend API systems, logs are often stored as plain text messages and exception handling is implemented using multiple try-catch blocks across different methods. This approach creates several problems such as unclear logs, code duplication, inconsistent error responses, and increased debugging time.

The main problem addressed in this project is the lack of a structured and centralized mechanism for logging and exception handling in ASP.NET Core backend APIs.

1.3 Objectives

- To study the limitations of traditional logging and exception handling approaches in backend APIs.
- To implement structured logging for better traceability and debugging.
- To implement centralized exception handling using middleware in ASP.NET Core.
- To reduce code duplication caused by repeated try-catch blocks.
- To improve maintainability and consistency of backend API responses.

2. Methodology

The methodology followed in this project involved designing two backend API approaches and comparing their behavior under different error scenarios. The first approach represented a traditional backend system, while the second approach represented the proposed improved system.

2.1 Traditional System

The traditional system used simple logging statements and multiple try-catch blocks inside controllers or service methods. Although this method can handle errors, it increases code repetition and makes the system difficult to maintain as the application grows.

2.2 Proposed System

The proposed system uses structured logging and centralized exception handling middleware. In this system, logs are stored in key-value format, making it easier to filter and analyze errors. Exception handling is moved to a centralized middleware component, which captures exceptions globally and returns standardized error responses.

2.3 Testing Scenarios

- Invalid input values were passed to APIs.
- Database failure scenarios were simulated.
- Unexpected API exceptions were generated.
- Responses and logs were compared between the traditional and proposed systems.

3. Tools and Technologies

- ASP.NET Core: Used for developing backend APIs.
- C#: Used as the primary programming language.
- Middleware: Used for centralized exception handling.
- Structured Logging: Used to store logs in readable key-value format.
- Visual Studio: Used as the development environment.
- SQL Database: Used for storing and retrieving application data.

4. Implementation

4.1 Structured Logging

Structured logging improves the quality of logs by storing information in a defined format instead of simple text messages. This allows developers to easily identify the time, request path, error type, and additional details related to an issue. It also improves log searching and filtering during debugging.

4.2 Centralized Exception Handling

Centralized exception handling was implemented using middleware in ASP.NET Core. Middleware captures errors globally before the response is returned to the client. This removes the need for writing repeated try-catch blocks in every controller method and ensures that all errors follow a consistent response format.

4.3 Proposed Flow

1. Client sends request to backend API.
2. Request passes through middleware pipeline.
3. API executes business logic and database operations.
4. If an exception occurs, middleware captures it globally.
5. Structured log entry is created with error details.
6. A consistent error response is returned to the client.

5. Major Findings/Outcomes/Output/Results

The project comparison showed that the proposed approach is more effective than the traditional approach for debugging and maintaining backend APIs.

Parameter	Traditional Approach	Proposed Approach
Logging	Simple and unstructured	Structured key-value logs
Exception Handling	Multiple try-catch blocks	Centralized middleware
Debugging Time	Higher	Reduced
Code Duplication	More duplication	Less duplication
Response Consistency	Varies by method	Standardized responses
Maintainability	Difficult as system grows	Improved and scalable

The results indicate that structured logging improves traceability and centralized exception handling reduces repeated code. Together, these techniques make the backend system cleaner, easier to debug, and more maintainable.

6. Conclusion and Future Scope

6.1 Conclusion

This project successfully demonstrates that structured logging and centralized exception handling can significantly improve debugging and maintainability in ASP.NET Core backend APIs. The traditional approach creates challenges due to scattered error handling and unclear logs, while the proposed approach provides cleaner code, better traceability, and consistent API responses.

By using middleware for centralized exception handling, developers can manage errors globally and reduce code duplication. Structured logging further improves the debugging process by storing useful error details in a clear format.

6.2 Future Scope

- The approach can be extended to microservices-based architectures.
- Distributed tracing can be added for tracking requests across multiple services.
- AI-based log analysis can be integrated to detect patterns and predict failures.
- Advanced monitoring dashboards can be developed for real-time backend health tracking.

References

1. Microsoft Documentation: ASP.NET Core Middleware.
2. Microsoft Documentation: Logging in .NET.
3. ASP.NET Core Web API development resources.
4. Study material and project presentation on structured logging and centralized exception handling.

Appendices

Appendix A: Project presentation slides were used as the base material for preparing this report.

Appendix B: Testing scenarios included invalid input, database failure, and API exception cases.